

컴퓨터비전개론_과제#5

1. YOLO v3를 이용하여 물체를 검출하는 프로그램을 수행한다. 'yolov3.weights'와 'coco_names.txt', 'yolov3.cfg' 파일은 Google drive에서 복사할 수 있다. 수행 프로그램은 다음 세 가지 중에서 하나를 사용하면 된다.

a. 오일석 교재 387~388쪽 프로그램을 사용한다. 이 프로그램은 OpenCV의 dnn 모듈을 사용하는 것인데, 프로그램 수행시 오류가 나타나므로 수정해서 사용하여야 한다.

b. Step by step YOLOv3-Object detection (kaggle.com)

c. How to Perform Object Detection With YOLOv3 in Keras - MachineLearningMastery.com

프로그램을 'soccer.jpg'와 'busy_street.jpg'에 적용하여 수행한 결과를 제시하라. 추가로 교내 도로에서 차량과 사람들이 포함된 영상을 취득하여 수행시킨 결과를 제시하라.

```
import numpy as np
import cv2 as cv
import sys

def construct_yolo_v3():
    with open('coco_names.txt', 'r') as f:
        class_names = [line.strip() for line in f.readlines()]

    model = cv.dnn.readNet('yolov3.weights', 'yolov3.cfg')
    model.setPreferableBackend(cv.dnn.DNN_BACKEND_DEFAULT)
    model.setPreferableTarget(cv.dnn.DNN_TARGET_CPU)

    layer_names = model.getLayerNames()
    out_layers = [layer_names[i - 1] for i in model.getUnconnectedOutLayers()]

    return model, out_layers, class_names

def yolo_detect(img, yolo_model, out_layers):
    height, width = img.shape[0], img.shape[1]
    test_img = cv.dnn.blobFromImage(img, 1.0/256, (448, 448), (0, 0, 0),
    swapRB=True)
    yolo_model.setInput(test_img)
    output3 = yolo_model.forward(out_layers)

    box, conf, id = [], [], []
    for output in output3:
        for vec85 in output:
            scores = vec85[5:]
            class_id = np.argmax(scores)
            confidence = scores[class_id]
            if confidence > 0.5:
                centerx, centery = int(vec85[0] * width), int(vec85[1] * height)
                w, h = int(vec85[2] * width), int(vec85[3] * height)
                x, y = int(centerx - w / 2), int(centery - h / 2)
```

```
        box.append([x, y, x + w, y + h])
        conf.append(float(confidence))
        id.append(class_id)

    ind = cv.dnn.NMSBoxes(box, conf, 0.5, 0.4)
    objects = [box[i] + [conf[i]] + [id[i]] for i in range(len(box)) if i in ind]
    return objects

# 모델 초기화
model, out_layers, class_names = construct_yolo_v3()
colors = np.random.uniform(0, 255, size=(len(class_names), 3))

# 이미지 목록
image_files = ['soccer.jpg', 'busy_street.jpg', 'school_car_person.jpg']

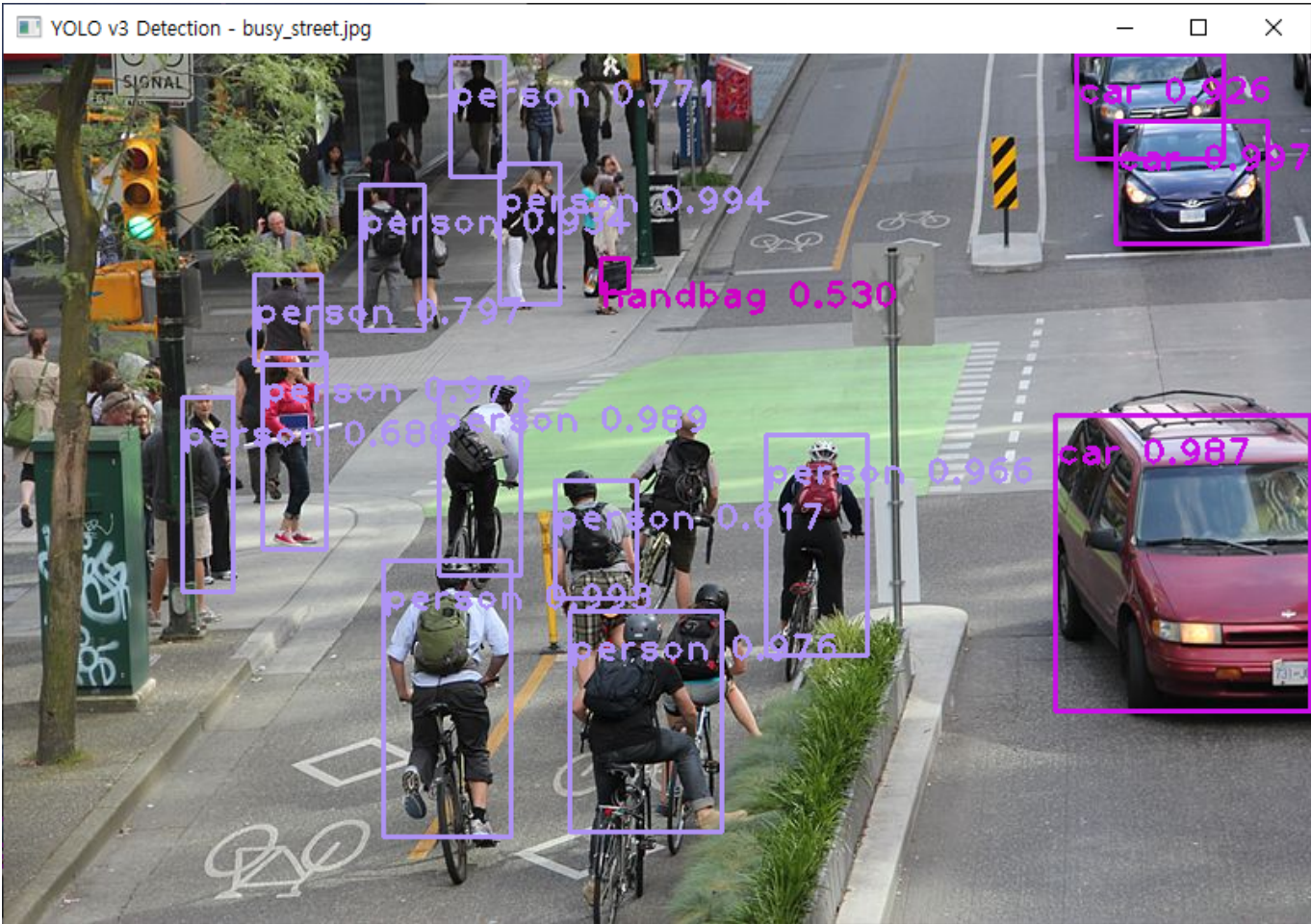
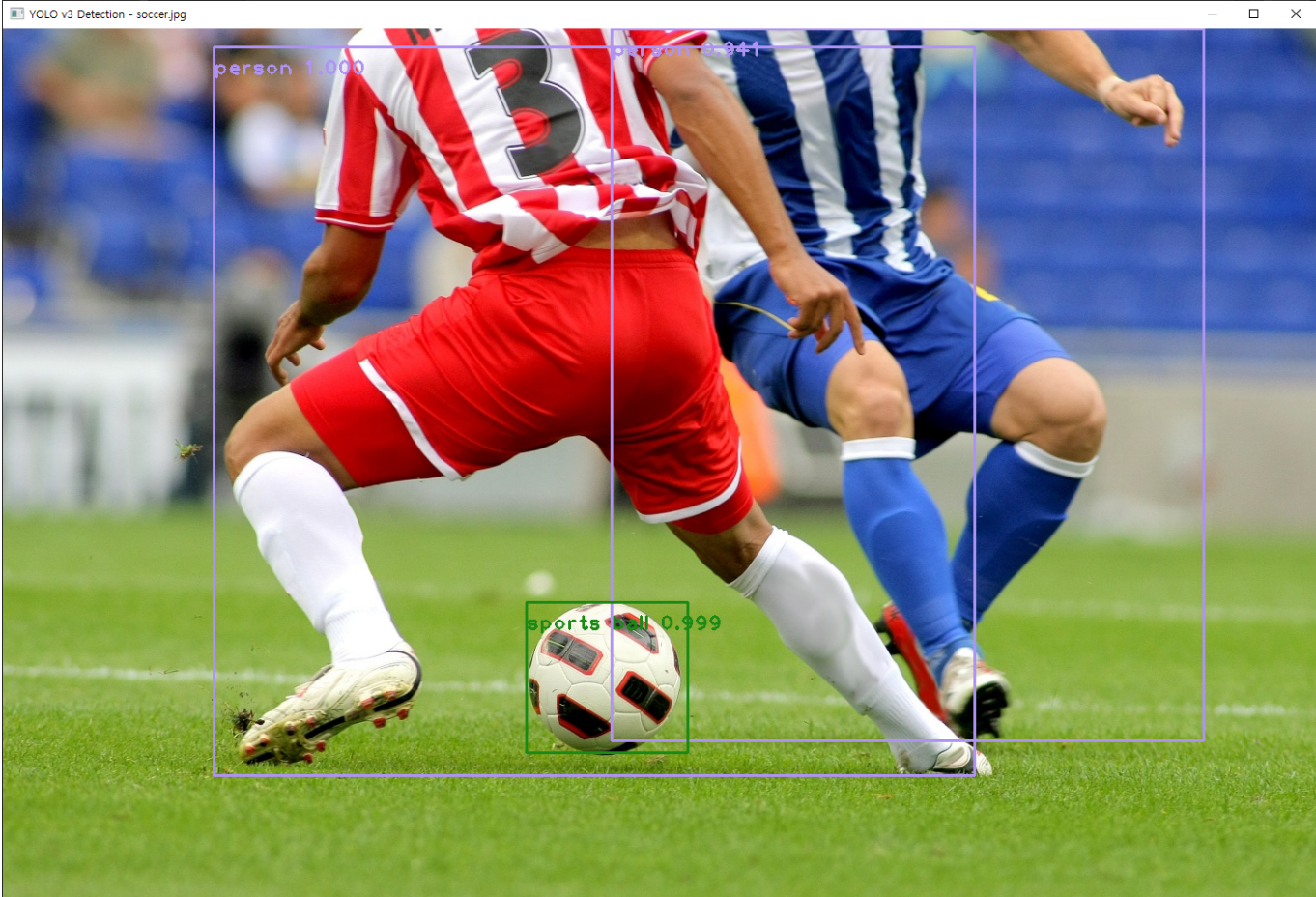
for image_file in image_files:
    img = cv.imread(image_file)
    if img is None:
        print(f'⚠ "{image_file}" 파일이 없습니다.')
        continue

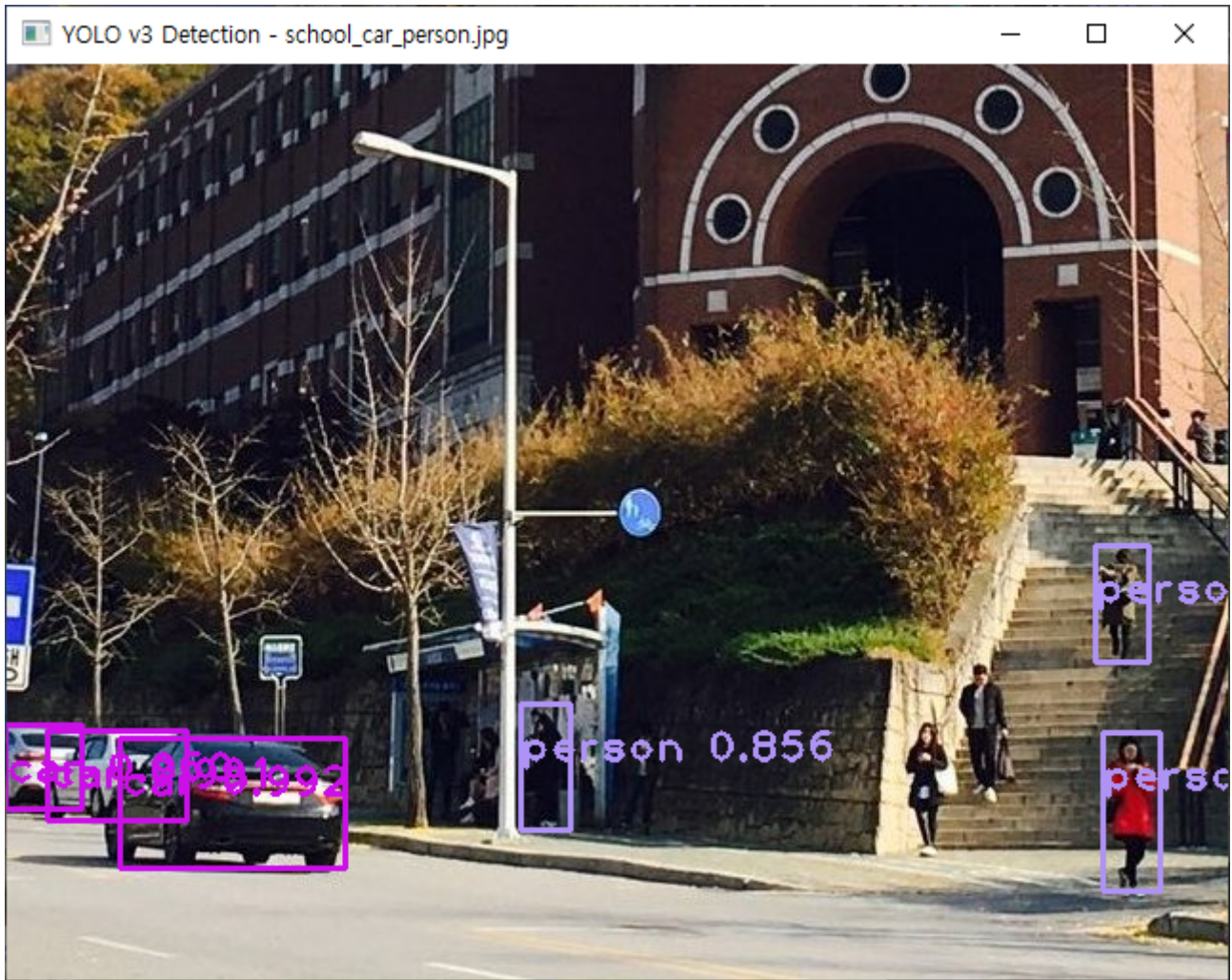
    res = yolo_detect(img, model, out_layers)

    for obj in res:
        x1, y1, x2, y2, confidence, class_id = obj
        text = f'{class_names[class_id]} {confidence:.3f}'
        cv.rectangle(img, (x1, y1), (x2, y2), colors[class_id], 2)
        cv.putText(img, text, (x1, y1 + 30), cv.FONT_HERSHEY_PLAIN, 1.5,
                   colors[class_id], 2)

# 이미지 출력
cv.imshow(f"YOLO v3 Detection - {image_file}", img)

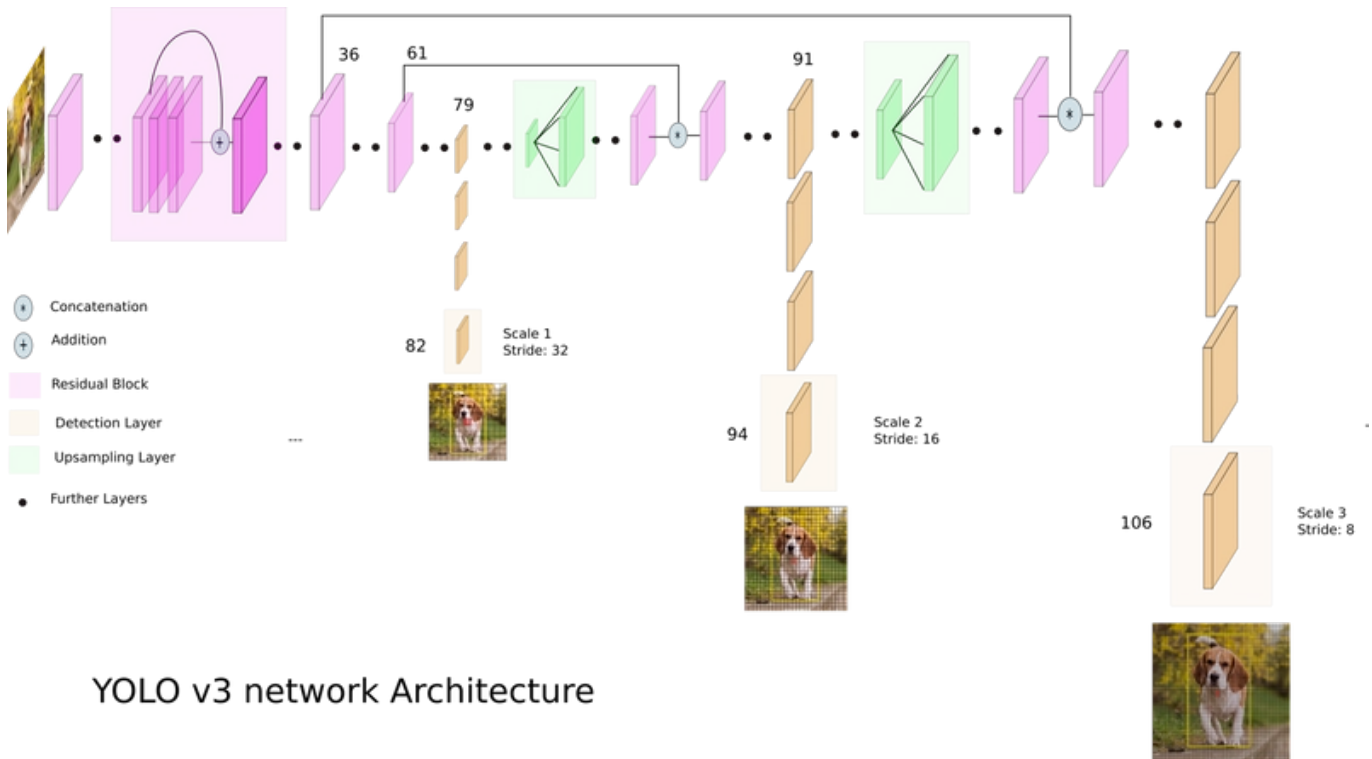
cv.waitKey(0)
cv.destroyAllWindows()
```





오일석 교재 387~388쪽의 프로그램을 오류 수정하여 코드 작성했습니다. 'soccer.jpg'에선 선수들과 공이 효과적으로 탐지되었으며, 각각에 대해 클래스명과 신뢰도가 표시되었고, 'busy_street.jpg'에선 혼잡한 도시 거리에서 다수의 보행자와 차량, 핸드백이 성공적으로 탐지되었습니다. 추가로, 교내에서 찍은 사진인 'school_car_person.jpg'에선 도서관 앞의 사람과 자동차가 인식되었고, 다양한 크기의 객체가 잘 처리되었습니다.

2. YOLO v3에서는 물체 검출을 어떻게 수행하는지 인터넷을 검색하여 조사하고 수행 절차를 그림을 이용하여 설명하라.



YOLO v3 network Architecture

위 다이어그램은 YOLOv3의 백본 네트워크인 Darknet-53의 예시입니다.

① 입력 이미지

- 이미지가 네트워크에 입력됩니다 (왼쪽의 강아지 이미지).
- 이 이미지는 여러 Conv 계층을 거치며 점점 더 작고 깊은 feature map으로 추상화됩니다.

② 특징 추출 (보라색 블록 = Residual Block)

- 입력 이미지는 Residual Block(보라색)들을 통해 처리됩니다.
- Residual Block은 잔차 연결(+)을 포함하여, 학습 안정성을 높이고 딥러닝 모델의 표현력을 향상시킵니다.
- 이 단계에서 점차 해상도는 줄고, 채널 수는 증가하게 됩니다.

③ 첫 번째 Detection Layer (노란색)

- 가장 깊은 특징맵(ex: 해상도 13x13)에서 첫 번째 물체 검출이 수행됩니다.
- 이 레이어는 큰 물체를 검출하는 데 적합합니다.
- 노란색 블록이 바로 Detection Layer를 의미합니다.

④ 업샘플링 및 병합 (초록색 + (*))

- 첫 검출 후, 업샘플링 레이어(초록색)를 통해 feature map을 2배 확대합니다.
- 확대된 feature map은 이전 단계에서 저장해둔 고해상도 feature map과 Concatenation(*) 됩니다.
- 이렇게 하여 고수준 특징 + 저수준 위치 정보가 결합됩니다.

⑤ 두 번째 Detection Layer

- 병합된 feature map(ex: 해상도 26x26)을 기반으로 중간 크기의 물체 검출이 수행됩니다.
- 다시 업샘플링 → 병합 → 검출 과정을 반복합니다.

⑥ 세 번째 Detection Layer

- 최종적으로 (ex: 해상도 52x52)인 feature map에서 작은 물체 검출이 수행됩니다.

⑦ 결과 통합 (Post-Processing)

- 세 가지 해상도(13x13, 26x26, 52x52)에서 얻은 박스 후보들에 대해
- 신뢰도 점수(Confidence score),
- 클래스 확률(Class probability),
- 비최대 억제(NMS) 등을 통해 최종 박스를 선택합니다.